

Momentum → RMSProp → Adam

Three ways to turn a noisy gradient into a useful update direction.

Momentum remembers direction. RMSProp remembers scale. Adam remembers both.

1 • Start with the optimizer contract

On minibatch $\mathcal{B}_t$, compute

$$\hat{g}_t = \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta_t).$$

Every optimizer constructs a direction d_t , then applies

$$\theta_{t+1} = \theta_t - \eta_t d_t.$$

The learning rate scales the move; the optimizer state changes its direction and coordinate-wise scale.

2 • EWMA is the shared memory primitive

$$m_t = \beta m_{t-1} + (1 - \beta)x_t, \quad 0 \leq \beta < 1.$$

Recent observations receive weights $(1 - \beta), (1 - \beta)\beta, \dots$

Effective memory: roughly $\frac{1}{1-\beta}$ observations.

Zero-start bias: early m_t is too small. Correct with $\hat{m}_t = \frac{m_t}{1-\beta^t}$.

3 • Momentum: consistent direction?

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \hat{g}_t$$

$$\theta_{t+1} = \theta_t - \eta m_{t+1}.$$

Signed gradients that keep the same sign reinforce one another; oscillating components cancel. When $\beta_1 = 0$, this normalized convention becomes ordinary GD.

PyTorch SGD uses an unnormalized momentum buffer, so its lr convention differs by a factor of $(1 - \beta_1)$ from the normalized rule above.

Ravine intuition

Momentum damps left-right reversals across steep walls while preserving progress along the shallow valley. Standardizing features may remove the ravine first.

4 • RMSProp: large relative to usual?

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) \hat{g}_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{g}_t}{\sqrt{v_{t+1}} + \varepsilon}$$

Squaring prevents sign cancellation. Dividing by the recent root-mean-square gives each coordinate its own ruler.

v is a second raw moment, not a centred variance. Operations are elementwise.

5 • Adam: direction and scale

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \hat{g}_t$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) \hat{g}_t^2$$

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \varepsilon}.$$

Bias correction matters most during the first updates, when both memories start at zero.

6 • What each method stores

Method	State	Question
GD	none	what is the gradient now?
Momentum	m	which direction persists?
RMSProp	v	which coordinates run large?
Adam	m, v	which direction, at what scale?

Adaptive does not mean parameter-free: tune the learning rate for the optimizer and problem.

Use adaptive optimizers deliberately

State initialization, step order, diagnostics, and fair comparisons.

1 • One correct Adam step

- 1 compute the current minibatch gradient
- 2 update first-moment state m
- 3 update second-moment state v
- 4 bias-correct using update count $t + 1$
- 5 divide $\hat{m}$ by $\sqrt{\hat{v} + \epsilon}$
- 6 apply the learning-rate-scaled parameter move

Initialize $m = 0$ and $v = 0$ once—not at every minibatch.

2 • Defaults are starting points

Knob	Typical	Effect
η	10^{-3} Adam	global step scale; tune first
β_1	0.9	longer signed-direction memory as it rises
β_2	0.999	longer squared-gradient memory as it rises
ϵ	10^{-8}	numerical floor; can affect scale in low precision

The useful learning-rate range depends on architecture, normalization, batch size, loss scale, and optimizer convention.

3 • PyTorch training order

```
opt.zero_grad(set_to_none=True)
pred = model(x)
loss = criterion(pred, y)
loss.backward()
opt.step()
```

Gradients accumulate unless cleared. The optimizer owns its per-parameter state; the model owns parameters and gradients.

AdamW note

AdamW applies weight decay as a separate shrink instead of mixing it into the adaptively scaled gradient. Treat the decay coefficient as a tuned regularization knob.

4 • Diagnose the symptom

Symptom

loss explodes / NaN

zigzag in a ravine

one coordinate dominates
no early progress

good train, poor validation

First checks

lower η ; inspect data, loss scale, logits, and gradients standardize; try momentum; reduce η check feature scale; consider RMSProp/Adam verify gradients/state; raise η carefully this is generalization—not an optimizer-only problem

5 • Compare optimizers fairly

- ✓ Same data split, shuffling policy, initialization, and model.
- ✓ Same update budget and logging frequency.
- ✓ Tune each optimizer's learning rate separately.
- ✓ Report validation performance, not only training loss.
- ✓ Run multiple seeds when differences are small.
- ✓ Log gradient norms, update norms, and optimizer state.

6 • Invariants worth asserting

Shapes: θ , g , m , and v match elementwise.

Finiteness: loss, gradients, and states contain no NaN/Inf.

Clock: bias correction uses the number of parameter updates.

State: checkpoint optimizer state when resuming training.

Optimizers change the path—not the objective.
Always inspect both training dynamics and validation behaviour.